



Namal University, Mianwali

Department of Computer Science

CSC-271-Database Systems

Project Proposal: FareShare Database System

Group Lead: Muhammad Naveed (NUM-BSCS-2024-54)

Member: Munawar Ali (NUM-BSCS-2024-60)

Member: Areeba Tahir (NUM-BSCS-2024-15)

Instructor: Ms. Asiya Batool

March 12, 2026

Contents

1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	System Scope and Feasibility	4
4.1	Functional Inclusions	4
4.2	Technical Constraints and Exclusions	4
5	Technology Stack	4
6	Detailed System Functionality Matrix	5
7	Task Division and Collaboration	5
7.1	Muhammad Naveed (Group Lead)	5
7.2	Munawar Ali	5
7.3	Areeba Tahir	5
8	Database Schema and Data Dictionary (Preliminary)	6
8.1	Proposed Relational Schema	6
9	Conclusion	6
10	References	6
A	AI Usage Disclosure	7

1 Introduction

FareShare is a specialized database system designed to revolutionize urban mobility in Pakistan, specifically within the Mianwali region. In alignment with the CSC-271 Database Systems curriculum, this project focuses on the design, implementation, and management of a high-integrity relational database. Unlike traditional ride-hailing services, FareShare introduces a native fare-sharing mechanism, which presents unique challenges in data modeling, concurrency control, and relational integrity.

The system is engineered to handle complex data relationships involving multiple riders sharing a single journey, necessitating sophisticated SQL querying and normalization techniques to ensure zero data redundancy and maximum operational efficiency.

2 Problem Statement

In the current socio-economic landscape of Mianwali, commuters lack a centralized, digitalized platform that supports localized, cash-optimized fare-sharing. Existing global platforms fail to address the specific needs of student populations and low-income workers who require dynamic cost-splitting.

From a database perspective, the problem lies in the inability of current systems to maintain consistent, non-redundant records of shared journeys where multiple entities interact with a single vehicle resource simultaneously. There is a critical need for a database architecture that can:

- Manage complex N-M (Many-to-Many) relationships between riders and shared rides.
- Ensure atomicity in fare calculations across multiple participating users.
- Maintain high-integrity logs for manual driver verification to bridge the existing security gap.

3 Objectives

The project is structured to demonstrate mastery of the Course Learning Outcomes (CLOs) defined in the CSC-271 syllabus:

1. **Logical Modeling:** To develop an Enhanced Entity-Relationship (EER) diagram that captures supertype/subtype relationships for different user classes (Riders, Drivers, Admins).
2. **Data Integrity:** To identify functional dependencies and apply normalization up to the Third Normal Form (3NF) to prevent insertion, deletion, and update anomalies.

3. **Physical Implementation:** To implement a physical database in MySQL using Data Definition Language (DDL) and Data Manipulation Language (DML).
4. **Advanced Querying:** To utilize nested subqueries, joins, and aggregate functions (GROUP BY, HAVING) to generate real-time revenue and ride analytics.

4 System Scope and Feasibility

4.1 Functional Inclusions

The database will serve as the backbone for the following modules:

- **User Authentication Subsystem:** Managing unique identifiers, mobile number verification (OTP logs), and role-based access control.
- **Ride Management Subsystem:** Tracking ride states (Created, Searching, Accepted, In-Progress, Completed).
- **Dynamic Fare Splitting Engine:** A relational implementation of fare formulas based on segment distance and vehicle type.
- **Safety and Audit Logs:** A dedicated subsystem for SOS alerts linked via Foreign Keys to the active ride and user tables.

4.2 Technical Constraints and Exclusions

To maintain feasibility within the semester timeline, the following are excluded from the database design:

- **Digital Payment Integration:** The database will strictly record cash transactions to avoid the complexity of third-party banking APIs.
- **Hardware-Level GPS Processing:** While geospatial coordinates are stored as data points, the system will not handle raw hardware signals, assuming pre-processed data via high-level language interfaces.

5 Technology Stack

In accordance with the weekly lab breakup, the system will utilize:

- **DBMS:** MySQL Server (Standard Relational DBMS).
- **Design Tools:** MySQL Workbench for EER Modeling and Schema Forward Engineering.
- **Query Language:** SQL (Structured Query Language) for all transactional logic.
- **Integration:** High-level language (Node.js/Python) for database connectivity.

6 Detailed System Functionality Matrix

Table 1: Database Functional Matrix

Module	Database Responsibility
Authentication	Store unique phone hashes, verify OTP timestamps, and manage JWT session tokens.
Driver Verification	Store document metadata (CNIC, License) and maintain a boolean 'is_verified' status.
Ride Matching	Execute geospatial queries to filter drivers based on current coordinates and vehicle types.
Fare Sharing	Calculate proportional splits using aggregate SQL functions and update multi-rider transaction logs.
Safety SOS	Immediate insertion of SOS records with high-priority indexing for administrative dashboards.

7 Task Division and Collaboration

The project tasks are distributed to ensure individual accountability as per the course policy:

7.1 Muhammad Naveed (Group Lead)

- Implementation of DDL scripts (Table creation, Constraints, Indexes).
- Development of Complex DQL scripts involving Multi-table Joins and Nested Sub-queries.
- Implementation of Database Triggers for automated logging and audit trails.

7.2 Munawar Ali

- Conversion of EER diagrams into Relational Schemas.
- Analysis of Functional Dependencies.
- Performing Normalization up to 3NF to ensure data integrity and remove anomalies.

7.3 Areeba Tahir

- Design of the Comprehensive EER Diagram.

- Implementation of Specialization and Generalization for User Subtypes.
- Defining Mapping Cardinalities and Participation Constraints (Strong vs. Weak Entities).

8 Database Schema and Data Dictionary (Preliminary)

The FareShare database will be centered around the following core relational structures:

8.1 Proposed Relational Schema

1. **Users**(UserID, PhoneNumber, FullName, Role, Rating, IsVerified)
2. **Vehicles**(VehicleID, *DriverID*, PlateNumber, Type, Capacity)
3. **Rides**(RideID, *PrimaryRiderID*, *DriverID*, StartTime, EndTime, IsShared, Status)
4. **FareDetails**(FareID, *RideID*, BaseFare, DistanceRate, TotalFare)

9 Conclusion

The FareShare database system is a rigorous application of relational database principles. By focusing on normalized structures and efficient SQL querying, the system will provide a scalable and secure foundation for carpooling in Mianwali. This project not only fulfills the CSC-271 semester requirements but also provides a practical solution to a real-world regional problem.

10 References

- *Modern Database Management*, Jeffrey A. Hoffer, V. Ramesh, Heikki Topi (13th Edition).
- FareShare Software Requirements Specification (SRS) v2.0.
- CSC-271 Course Outline, Department of Computer Science, Namal University.
- GitHub Repository: <https://github.com/Naveed83067/FareShare-Ride-Sharing-System.git>.

A AI Usage Disclosure

- **AI Tool:** Gemini 3 Flash.
- **Purpose:** Generating LaTeX boilerplate, aligning SRS deliverables with CSC-271 CLOs, and expanding technical descriptions.
- **Access Date:** March 11, 2026.